

Task 14 – Ensemble Methods: Bagging & Boosting

Dataset: Breast Cancer Wisconsin (Diagnostic) — Binary Classification

Part A – Dataset Selection & Preparation

A1. Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset was selected for this task. It is a built-in scikit-learn dataset, requiring no external download, and represents a realistic binary medical diagnosis problem.

- Samples: 569
- Features: 30 numeric features computed from digitized images of a breast mass (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension — each given as mean, standard error, and "worst" value)
- Target variable: binary — 0 = malignant, 1 = benign
- Class balance: 212 malignant, 357 benign (moderately imbalanced but manageable)



Figure: Class distribution of the target variable.

A2. Preprocessing & Split

No missing values were present in the dataset. Because features are on very different numeric scales (e.g. area vs. smoothness), StandardScaler was applied to standardize all features to zero mean and unit variance. The data was split into 80% training and 20% testing sets, using stratified sampling on the target variable to preserve class balance in both sets.

Split	Samples	Malignant %	Benign %
Training	455	37.3%	62.7%
Testing	114	37.3%	62.7%

Part B – Bagging

B1. Implementation

A Bagging Classifier was implemented using scikit-learn's BaggingClassifier with a Decision Tree as the base estimator (100 estimators, bootstrap sampling enabled). This is the classic Bagging setup — an ensemble of independently trained trees, each trained on a bootstrap sample of the training data, combined via majority voting.

B2. Evaluation Results

Metric	Value
Accuracy	0.9386
Precision	0.9452
Recall	0.9583
F1-Score	0.9517

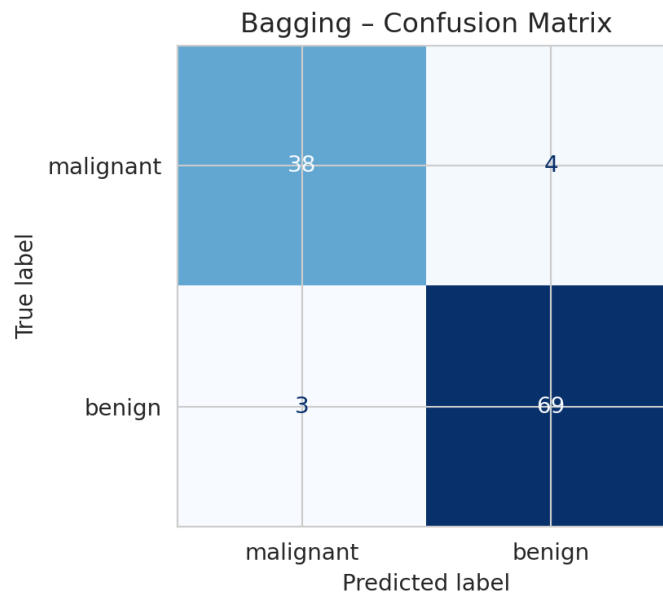


Figure: Confusion matrix for the Bagging classifier.

B3. Advantages, Limitations & Use Cases

Advantages

- Reduces variance and overfitting by averaging many high-variance base learners (e.g. deep decision trees).
- Robust to noisy data and outliers, since predictions are aggregated via majority vote.
- Base learners can be trained in parallel, making it fast on multi-core machines.
- Simple to implement and tune — the main hyperparameter is the number of estimators.

Limitations

- Does little to reduce bias — a systematically wrong base learner remains biased even after bagging.
- Less interpretable than a single decision tree.
- Requires more memory/storage since many models are kept.
- Diminishing returns beyond a certain number of estimators.

Use Cases

- High-variance, low-bias base models such as unpruned decision trees.
- Tabular data problems with noisy features — such as this breast cancer diagnosis task.
- Random Forest is the most well-known extension of Bagging.

Part C – Boosting

C1. Implementation

An XGBoost classifier was implemented (100 estimators, max depth 4, learning rate 0.1). Unlike Bagging, Boosting builds trees sequentially, where each new tree focuses on correcting the errors of the previous ones.

C2. Evaluation Results

Metric	Value
Accuracy	0.9474
Precision	0.9459
Recall	0.9722
F1-Score	0.9589

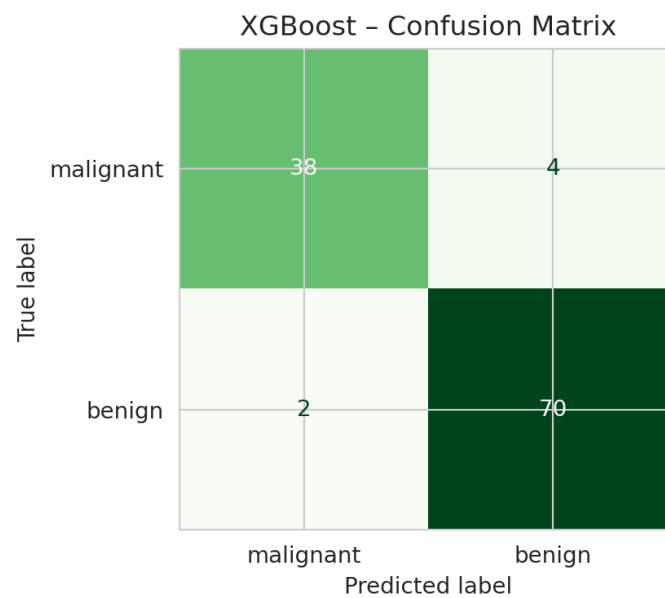


Figure: Confusion matrix for the XGBoost classifier.

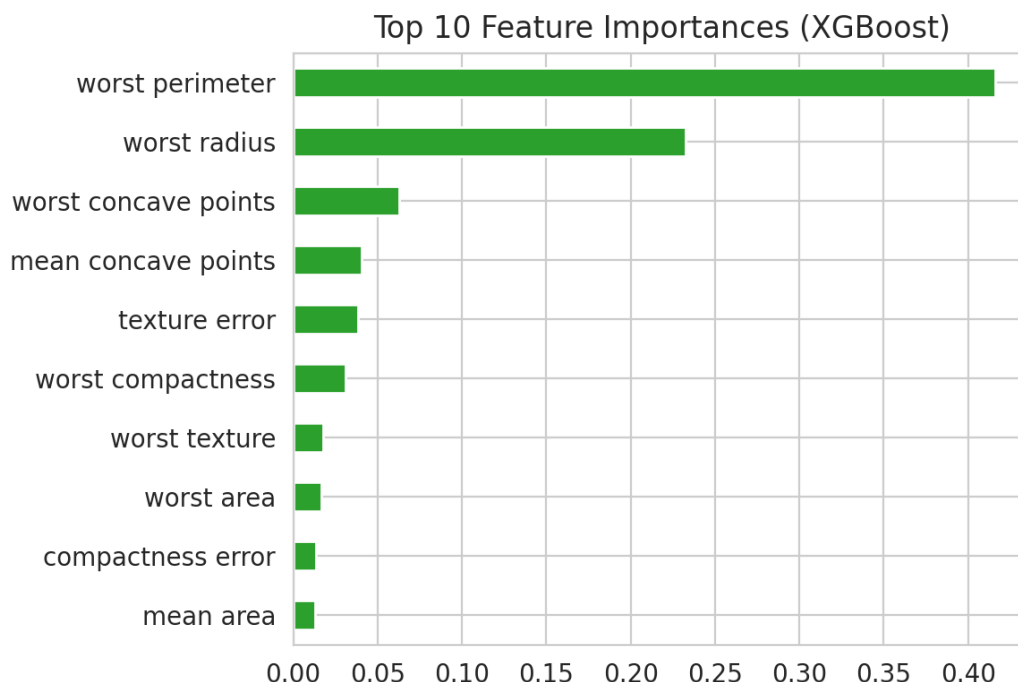


Figure: Top 10 feature importances from XGBoost.

C3. Comparison with Bagging

Model	Accuracy	Precision	Recall	F1-Score	Train Time (s)
Bagging	0.9386	0.9452	0.9583	0.9517	0.537
XGBoost	0.9474	0.9459	0.9722	0.9589	0.097

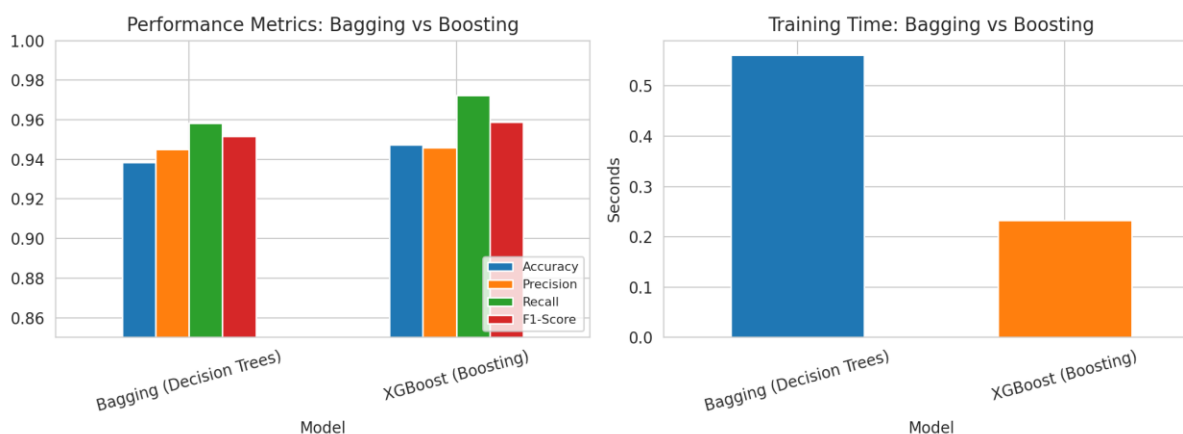


Figure: Performance metrics and training time — Bagging vs Boosting.

On this dataset, XGBoost achieved slightly higher accuracy, precision, recall, and F1-score than Bagging, and trained faster — its trees are shallow (max depth 4) and the implementation is highly optimized, whereas Bagging trains 100 full-depth trees.

C4. Advantages, Limitations & Applications of Boosting

Advantages

- Reduces both bias and variance — sequential error-correction typically yields higher accuracy than Bagging.
- Often achieves state-of-the-art performance on structured/tabular data.
- Built-in regularization (in XGBoost/LightGBM) helps control overfitting.
- Provides useful feature importance scores.

Limitations

- Sequential training cannot be parallelized across estimators, unlike Bagging.
- More sensitive to noisy data and outliers, since it repeatedly focuses on hard-to-classify examples.
- More hyperparameters to tune (learning rate, tree depth, number of estimators, regularization) — higher risk of overfitting without careful tuning.
- Less interpretable ("black box"), though tools like SHAP can help.

Applications

- Credit scoring, fraud detection, medical diagnosis, click-through-rate prediction, ranking systems — widely used in industry and ML competitions (e.g. Kaggle) for tabular data.

Part D – Comparative Analysis & Recommendation

D1. Bagging vs Boosting

Criterion	Bagging	Boosting (XGBoost)
Performance (this dataset)	Accuracy 0.939, F1 0.952	Accuracy 0.947, F1 0.959 (slightly better)
Bias/Variance	Reduces variance only	Reduces both bias and variance
Training style	Parallel (independent trees)	Sequential (each tree depends on the last)
Computational efficiency	Slower here (100 full-depth trees); trivially parallelizable	Faster here (shallow trees, optimized library); not parallel across rounds
Overfitting risk	Lower — averaging stabilizes predictions	Higher — needs careful tuning
Sensitivity to noise/outliers	More robust	More sensitive
Interpretability	Moderate	Lower (feature importance / SHAP available)
Tuning effort	Low	Higher

D2. Recommendation

For this dataset (breast cancer diagnosis), Boosting (XGBoost) is the recommended method:

- It achieved higher accuracy, precision, recall, and F1-score than Bagging in the experiments above.
- In a medical diagnosis context, recall on the malignant class matters greatly (missing a malignant case is costly), and XGBoost showed a stronger recall/F1 balance.
- The dataset is clean (no significant outliers or missing data), which suits Boosting well since its main weakness — sensitivity to noise — is not a major concern here.
- The dataset is small (569 samples, 30 features), so XGBoost's higher tuning effort is manageable and training time is not a bottleneck for either method.

Bagging would be preferred instead if the data were noisier, if fast parallel training on very large datasets were required, or if a simpler, more robust, low-maintenance baseline model were desired.

Summary

Metric	Bagging	XGBoost (Boosting)
Accuracy	0.939	0.947
Precision	0.945	0.946
Recall	0.958	0.972
F1-Score	0.952	0.959

Both ensemble methods performed strongly on this dataset, confirming that ensembling improves upon a single decision tree. Boosting (XGBoost) outperformed Bagging on every metric, making it the recommended approach for this classification task.